

Frontend Dev Priorities — Absa Customer Lifecycle Demo

For: Frontend Dev Team · August 6, 2026

Context: Demo tomorrow with Absa Bank Zambia. Backend APIs are live on <http://100.82.12.85>. I will handle all API wiring — you focus on UI components, state management, and visual polish.

Reference: [docs/FRONTEND-REQUIREMENTS-V2.md](#) for full specs.

📖 Page Map — What Goes Where

PAGE	ROUTE	COMPONENTS
Dashboard Home	/dashboard/home	
└ KPI Cards	(inline)	StateBadge
└ Customer Table	(inline)	StateBadge
Customer Detail	/dashboard/customer/:id	
└ Profile Card	(inline)	
└ Health Gauge	HealthScoreGauge	
└ Churn Bar	ChurnProbabilityBar	
└ State Timeline	StateTimeline	
└ Markov Matrix	MarkovMatrix	
Portfolio Overview	/dashboard/portfolio	
└ KPI Cards	(inline)	
└ State Pie Chart	(inline, vue-chartjs)	
└ Health Histogram	(inline, vue-chartjs)	
ETL Pipeline	/dashboard/etl-pipeline	
└ Run Table	(inline)	
ETL Run History	/dashboard/etl-run-history	
└ Audit Table	(inline)	
Layout (all pages)	DashboardLayout.vue	
└ Sidebar	(cleaned to 6 items)	
└ Period Selector	(new, header)	

What I Need From You (Before Tomorrow)

PAGE 1: Dashboard Home — [src/views/DashboardHome.vue](#)

Route: `/dashboard/home`
This is the landing page after login.

What's there now:

4 hardcoded KPI cards (1,240, 68, 42, 12) + a customer table with mock data.

What you need to do:

1. KPI Cards — make reactive Replace all hardcoded numbers with `{{ }}` bindings so I can plug in real data:

Card	Replace This	With This
Total Customers	1,240	<code>{{ portfolio.total }}</code>
At Risk	68 5.5%	<code>{{ portfolio.atRisk }}</code> <code>{{ portfolio.atRiskPct }}</code> %
Dormant	42 3.4%	<code>{{ portfolio.dormant }}</code> <code>{{ portfolio.dormantPct }}</code> %
Actions Due	12	<code>{{ portfolio.actionsDue }}</code>

Remove the hardcoded trend indicators (+2.4%, +4 since last snapshot, Stable across 3 periods).

2. KPI Cards — add click handlers Each card should filter the customer table below when clicked:

```
Click "At Risk"    → setFilter('state', 'AT_RISK')
Click "Dormant"    → setFilter('state', 'DORMANT')
Click "Total"      → clearFilters()
```

3. Customer Table — add StateBadge Import and use `<StateBadge>` in the State column of every row:

```
<StateBadge :state="customer.state" size="sm" />
```

4. Add loading skeleton Show skeleton cards + skeleton table rows while `customerStore.loading === true`.

5. Add error banner If `customerStore.error`, show red banner: "Could not load portfolio data" + [Retry] button.

PAGE 2: Customer Detail — `src/views/CustomerDetail.vue`

Route: `/dashboard/customer/:id`
⚠ THIS IS THE DEMO CENTERPIECE — most important page.

What's there now:

Entirely hardcoded — "Mwenda Kapambwe (ID: 994022/11/1)", fake profile card, no AI data.

What you need to do:

1. Profile Card (top-left section) — reactive bindings

Replace ALL hardcoded text:

```
"Mwenda Kapambwe"      → {{ customer.fullName || 'Loading...' }}
"994022/11/1"          → {{ customer.customerId }}
"Verified Private Client" → {{ customer.segment }}
```

Keep the red ABSA header bar and avatar circle styling — just replace the data.

2. Insert HealthScoreGauge — below the profile card

```
<HealthScoreGauge
  :score="healthScore"
  :trend="healthTrend"
  :previousScore="previousHealthScore"
/>
```

Place it right below the customer info section, before the AI Intelligence section.

3. Insert ChurnProbabilityBar — next to the health gauge

```
<ChurnProbabilityBar
  :probability="churnProbability"
  :showLabel="true"
/>
```

Place it in the same row as the health gauge or directly below it.

4. Insert StateTimeline — in the AI Intelligence section

```
<StateTimeline :transitions="timeline" />
```

This replaces whatever placeholder content is in the AI hub area. The timeline shows the customer's journey: Active → At Risk → Dormant → Churned.

5. Insert MarkovMatrix — in a collapsible section below the timeline

```
<details>
  <summary>Advanced: State Transition Probabilities</summary>
  <MarkovMatrix :matrix="markovMatrix" />
</details>
```

Use `<details>` / `<summary>` native HTML element for the collapsible — simple, no JS needed.

6. Loading state While `customerStore.loading` is true, show:

- Profile card: grey blocks (skeleton)
- Health gauge area: spinning ring
- Timeline area: 4 grey dots in a row
- Markov matrix: hidden (only show when data exists)

7. Error state If `customerStore.error`:

- Red banner at top: "Could not load customer data"
- [Retry] button

8. Empty state (no predictions) If customer data loads but health/churn is null:

- Amber banner: "Predictions not yet computed for this customer. Run batch prediction to generate health score and churn probability."

PAGE 3: Portfolio Overview — `src/views/PortfolioOverview.vue`

Route: `/dashboard/portfolio`

What's there now:

KPI cards similar to DashboardHome, with hardcoded values.

What you need to do:

1. KPI Cards — reactive (same treatment as DashboardHome)

```
Total Customers    → {{ portfolio.total }}
At Risk            → {{ portfolio.atRisk }}
Dormant            → {{ portfolio.dormant }}
Churned (NEW)      → {{ portfolio.churned }}
```

2. Add State Distribution pie/donut chart Use `vue-chartjs` (already vendored in the project). Create an inline `<Doughnut>` chart showing Active / At Risk / Dormant / Churned proportions. Colors: Green, Amber, Grey, Red.

Place it in the main content area, taking ~40% width, next to the histogram.

3. Add Health Score histogram placeholder Use `vue-chartjs` `<Bar>` chart. Placeholder with axes labeled — I'll connect the data. Place it next to the pie chart (~60% width).

4. Loading / Error / Empty states — same pattern as DashboardHome.

PAGE 4: ETL Run History — `src/views/Modules/datapipeline/ETLRunHistory.vue`

Route: `/dashboard/etl-run-history`

What's there now:

Already wired to `etlApi.js` — may just need polish.

What you need to do:

1. Status badges Add colored status badges in the table:

- COMPLETED → green badge with check
- FAILED → red badge with X
- RUNNING → blue badge with spinner

2. Add loading/error/empty states (same pattern as above).

3. Make rows clickable — clicking a run row should navigate to `/dashboard/etl-run-history/batch/:runId` (route already exists as `BatchExecutionDetail`).

PAGE 5: ETL Pipeline — `src/views/Modules/datapipeline/EtlPipeline.vue`

Route: `/dashboard/etl-pipeline`

What you need to do:

1. Polish the existing view — loading/error/empty states, clean table styling consistent with other pages.

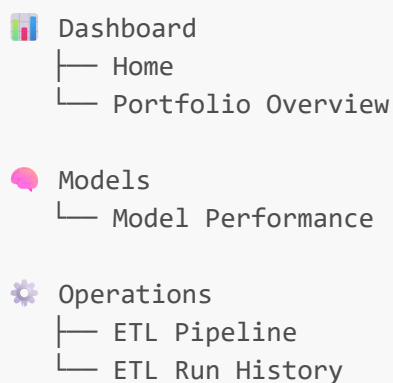
LAYOUT: DashboardLayout — `src/components/layouts/DashboardLayout.vue`

Used by: ALL dashboard pages

What you need to do:

1. Clean the sidebar — only show 6 items for demo

Current sidebar has 30+ items (CRM, Strategic Management, AI Module, etc.). For the demo, only show:



Do this by editing the sidebar template in `DashboardLayout.vue` — comment out the other items, don't delete. Use `v-if="false"` or `<!-- -->` comments.

2. Add Period Selector dropdown to the header

```
<div class="period-selector">
  <span>Data Snapshot:</span>
  <select v-model="selectedPeriod">
    <option value="latest">Latest (August 6, 2026)</option>
  </select>
</div>
```

Place it in the top-right area of the header. I'll populate the options dynamically later.

ROUTER: Navigation Cleanup — `src/router/index.js`

1. No changes needed to the router itself — just the sidebar template in `DashboardLayout.vue` controls what's visible. The routes stay defined.

Components to Create (all in `src/components/absa/`)

Component	File	Used On
HealthScoreGauge	<code>src/components/absa/HealthScoreGauge.vue</code>	CustomerDetail
StateBadge	<code>src/components/absa/StateBadge.vue</code>	DashboardHome, CustomerDetail, PortfolioOverview
StateTimeline	<code>src/components/absa/StateTimeline.vue</code>	CustomerDetail
MarkovMatrix	<code>src/components/absa/MarkovMatrix.vue</code>	CustomerDetail
ChurnProbabilityBar	<code>src/components/absa/ChurnProbabilityBar.vue</code>	CustomerDetail

Component Specs

HealthScoreGauge — Used on: CustomerDetail (below profile card)

Circular gauge, 0-100.

Props:

score

Number

0-100

trend

String

'up' | 'down' | 'stable'

previousScore

Number

for trend arrow calculation

States:

loading

→ grey skeleton ring (pulsing)

```
error      → broken ring icon + "Failed to load" + [Retry] button
empty      → grey ring, "No data" text centered
loaded     → colored ring with score number in center
```

Colors:

```
Green  (#00873E)  score >= 70
Amber  (#FFB300)  score 40-69
Red    (#77021E)  score < 40
```

Size: ~180px diameter – large enough to be the focal point on CustomerDetail

StateBadge — Used on: DashboardHome (table), CustomerDetail (profile), PortfolioOverview (chart legend)

Pill/badge showing lifecycle state.

Props:

```
state  String  'ACTIVE' | 'AT_RISK' | 'DORMANT' | 'CHURNED'
size   String  'sm' (table rows) | 'md' (cards) | 'lg' (detail header)
```

Display:

```
ACTIVE    →  "Active"      bg: #E8F5E9 text: #00873E
AT_RISK   →  "At Risk"     bg: #FFF8E1 text: #E65100
DORMANT   →  "Dormant"     bg: #F5F5F5 text: #616161
CHURNED   →  "Churned"     bg: #FFEBEE text: #77021E
```

Include a small circle/dot icon inside the badge, matching the color.

StateTimeline — Used on: CustomerDetail (AI Intelligence section)

Horizontal timeline of state changes.

Props:

transitions `Array<{from, to, date, daysInState, triggerReason}>`

Display:

Left-to-right horizontal line with colored dots at each transition point.

Dot color = the "to" state color.

Lines between dots = grey.

Current state dot = larger, with pulsing ring.

Hover on dot → tooltip: "Mar 15 – DORMANT: 90 days inactivity"

States:

loading → 4 grey dots in a row (skeleton)

empty → "No state transitions recorded" centered text

error → "Failed to load timeline" + [Retry]

MarkovMatrix — Used on: CustomerDetail (collapsible section below timeline)

4×4 probability heatmap.

Props:

matrix `Array<Array<Number>>` 4×4 grid of probabilities

states `Array<String>` ['ACTIVE', 'AT_RISK', 'DORMANT', 'CHURNED']

Display:

Table with row headers (FROM) and column headers (TO).

Each cell = probability formatted as "0.82".

Background color intensity: white (0.00) → red (#77021E at 1.00).

Row with all 0s (e.g., CHURNED row) → greyed out with "absorbing" note.

States:

loading → 4×4 skeleton grid

empty → "No transition data available"

error → "Failed to load matrix" + [Retry]

ChurnProbabilityBar — Used on: CustomerDetail (next to health gauge)

Horizontal progress bar.

Props:

probability `Number` 0.00 - 1.00

showLabel `Boolean` default true

Display:

Horizontal bar, width proportional to probability.

Label: "Churn Risk: 18%" or "Churn Risk: 82%"

Colors:

Green (#00873E) < 0.30
Amber (#FFB300) 0.30 - 0.60
Red (#77021E) > 0.60

Width: ~300px, height: ~24px.

Pinia Stores — Create Structure (I fill actions)

src/stores/customerStore.js — Used by: DashboardHome, CustomerDetail, PortfolioOverview

```
import { defineStore } from 'pinia'

export const useCustomerStore = defineStore('customer', () => {
  // State – define these
  const customers = ref([])
  const selectedCustomer = ref(null)
  const filters = ref({ state: null, search: '', branch: null })
  const pagination = ref({ page: 1, limit: 25, total: 0 })
  const loading = ref(false)
  const error = ref(null)

  // Actions – define signatures, I'll fill bodies
  async function fetchPortfolio(params) { /* I will implement */ }
  async function fetchCustomerDetail(id) { /* I will implement */ }
  async function fetchCustomerTimeline(id) { /* I will implement */ }
  function setFilter(key, value) { /* I will implement */ }
  function clearFilters() { /* I will implement */ }

  return { customers, selectedCustomer, filters, pagination, loading, error,
    fetchPortfolio, fetchCustomerDetail, fetchCustomerTimeline, setFilter,
    clearFilters }
})
```

src/stores/predictionStore.js — Used by: CustomerDetail

```
import { defineStore } from 'pinia'

export const usePredictionStore = defineStore('prediction', () => {
  const predictions = ref({})
  const healthScores = ref({})
  const markovMatrix = ref(null)
  const loading = ref(false)
  const error = ref(null)

  async function fetchChurnProbability(customerId) { /* I will implement */ }
```

```
async function fetchHealthScore(customerId) { /* I will implement */ }
async function fetchMarkovMatrix() { /* I will implement */ }

return { predictions, healthScores, markovMatrix, loading, error,
         fetchChurnProbability, fetchHealthScore, fetchMarkovMatrix }
})
```

State Management on Buttons — All pages

Every action button needs 3 states. Apply this pattern everywhere:

```
<script setup>
const saving = ref(false)
const saved = ref(false)
const saveError = ref(null)

async function handleSave() {
  saving.value = true
  saved.value = false
  saveError.value = null
  try {
    await someAction()
    saved.value = true
    setTimeout(() => (saved.value = false), 2000)
  } catch (e) {
    saveError.value = e.message
  } finally {
    saving.value = false
  }
}
</script>

<template>
  <button :disabled="saving" @click="handleSave">
    <span v-if="saving"><SpinnerIcon /> Saving...</span>
    <span v-else-if="saved"><CheckIcon /> Saved</span>
    <span v-else>Save</span>
  </button>
  <p v-if="saveError" class="error">{{ saveError }}</p>
</template>
```

What I Will Handle

Task	Owner
<code>customerApi.js</code> — all backend API calls	Me
<code>predictionApi.js</code> — churn/health/markov endpoints	Me

Task	Owner
Filling in Pinia store actions with real API calls	Me
Auth flow testing against 100.82.12.85:8080	Me
Wiring components to stores (passing props)	Me
CORS/network debugging	Me

File Checklist

CREATE:

- ☐ src/components/absa/HealthScoreGauge.vue → CustomerDetail page
- ☐ src/components/absa/StateBadge.vue → DashboardHome, CustomerDetail, PortfolioOverview
- ☐ src/components/absa/StateTimeline.vue → CustomerDetail page
- ☐ src/components/absa/MarkovMatrix.vue → CustomerDetail page
- ☐ src/components/absa/ChurnProbabilityBar.vue → CustomerDetail page
- ☐ src/stores/customerStore.js → all pages (structure only)
- ☐ src/stores/predictionStore.js → CustomerDetail page (structure only)

EDIT:

- ☐ src/views/DashboardHome.vue → reactive KPI cards, StateBadge, loading/error
- ☐ src/views/CustomerDetail.vue → reactive profile, 4 new components, loading/error/empty
- ☐ src/views/PortfolioOverview.vue → reactive KPIs, pie chart, histogram placeholder
- ☐ src/views/Modules/datapipeline/ETLRunHistory.vue → status badges, loading/error/empty
- ☐ src/views/Modules/datapipeline/EtlPipeline.vue → polish, loading/error/empty
- ☐ src/components/layouts/DashboardLayout.vue → cleaned sidebar (6 items), period selector

Backend APIs Available (For Reference)

All on <http://100.82.12.85>. Individual services: 8002 (features), 8003 (state), 8004 (prediction), 8080 (gateway).

Endpoint	Used By Page	Returns
GET /states/portfolio? as_of_date=...	DashboardHome, PortfolioOverview	{ total, active, at_risk, dormant, churned }
GET /states/{id}? as_of_date=...	CustomerDetail	{ customer_id, state, health_score, component_scores }

Endpoint	Used By Page	Returns
GET /states/{id}/timeline	CustomerDetail	{ transitions: [{from, to, date, trigger_reason}] }
GET /states/markov/matrix?as_of_date=...	CustomerDetail	{ states, matrix: 4x4 }
GET /predict/{id}/churn?as_of_date=...	CustomerDetail	{ churn_probability, model_version }
GET /predict/{id}/health?as_of_date=...	CustomerDetail	{ health_score, component_scores }
GET /api/etl/runs?page=1&limit=25	ETLRunHistory, EtlPipeline	{ kpis, runs[], total_runs }

Questions?

Ask in discord. let me know when components are ready and I'll wire them to the backend.